

دانشکده مهندسی کامپیوتر

طراحی و پیاده‌سازی چارچوب تست E2E برای نرم‌افزارهای مبتنی بر Qt C++

پایان‌نامه کارشناسی

در رشته مهندسی کامپیوتر

علیرضا بیات خانی

۹۹۵۲۱۱۳۶

استاد راهنما:

دکتر ابوالفضل دیانت

پاییز ۱۴۰۵


بسم الله الرحمن الرحيم

تأییدیه هیئت داوران جلسه دفاع از پایان‌نامه / رساله

نام دانشکده: دانشکده مهندسی کامپیوتر

نام دانشجو: علیرضا بیات خانی

عنوان پایان‌نامه: طراحی و پیاده‌سازی چارچوب تست E2E برای نرم‌افزارهای مبتنی بر Qt C++

تاریخ دفاع: پاییز ۱۴۰۵

رشته: در رشته مهندسی کامپیوتر

گرایش:

ردیف	سمت	نام و نام‌خانوادگی	دانشگاه یا مؤسسه	امضا
۱	استاد راهنما	دکتر ابوالفضل دیانت	دانشگاه علم و صنعت ایران	
۲	استاد مدعو خارجی	دکتر	دانشگاه	
۳	استاد مدعو داخلی	دکتر	دانشگاه علم و صنعت ایران	

تأییدیه صحت و اصالت نتایج

باسمه تعالی

این‌جانب **علیرضا بیات خانی** به شماره دانشجویی **۹۹۵۲۱۱۳۶** دانشجو رشته **مهندسی کامپیوتر** مقطع تحصیلی **کارشناسی** تایید می‌نمایم که کلیه مندرجات در این پایان‌نامه حاصل کار پژوهشی اینجانب تحت نظارت و راهنمایی عضو هیأت علمی دانشگاه علم و صنعت ایران، بدون هرگونه دخل و تصرف انجام گرفته و موارد نسخه‌برداری شده از آثار دیگران، مطابق مقررات و ضوابط ارجاع داده شده و ویژگی‌های کامل منابع را در فهرست منابع ذکر کرده‌ام. این پایان‌نامه پیش‌تر برای احراز هیچ مدرکی ارائه نگردیده است.

در صورت اثبات خلاف مندرجات فوق، به تشخیص دانشگاه مطابق با ضوابط و مقررات حاکم (قانون حمایت از حقوق مولفان و منصفان و قانون ترجمه، تکثیر و نشریات و آثار صوتی، ضوابط و مقررات آموزشی و پژوهشی، انضباطی و غیره) با اینجانب رفتار خواهد شد و حق هرگونه اعتراض در خصوص احقاق حقوق مکتسب و تشخیص و تعیین تخلف و مجازات را از خویش سلب می‌نمایم. در ضمن، مسئولیت هرگونه پاسخگویی به اشخاص اعم از حقیقی و حقوقی و مراجع ذی‌صلاح (اعم از اداری و قضایی) به عهده اینجانب خواهد بود و دانشگاه هیچ گونه مسئولیتی در این خصوص نخواهد داشت.

کلیه نتایج و حقوق حاصل از این پایان‌نامه متعلق به دانشگاه علم و صنعت ایران است. هرگونه استفاده از نتایج علمی و عملی و واگذاری اطلاعات به دیگران یا چاپ و تکثیر، نسخه‌برداری، ترجمه و اقتباس از این پایان‌نامه بدون موافقت کتبی دانشگاه علم و صنعت ایران ممنوع است. نقل مطالب با ذکر منبع بلامانع است.

نام و نام‌خانوادگی دانشجو: علیرضا بیات خانی
تاریخ: پاییز ۱۴۰۵
امضای دانشجو:

چکیده

تست خودکار رابط کاربری برای نرم‌افزارهای مبتنی بر Qt C++ تا امروز عمدتاً به ابزارهای تجاری محدود بوده و ابزارهای متن‌باز موجود، تنها زیرساخت خام (درایور و پروتکل) را فراهم می‌کنند. این پایان‌نامه طراحی و پیاده‌سازی یک چارچوب تست سرتاسری E2E[1] متن‌باز برای برنامه‌های دسکتاپ مبتنی بر Qt در بستر ویندوز را گزارش می‌کند. چارچوب بر زیرساخت Appium/WinAppDriver استوار است و با پنج لایه‌ی معماری — مکان‌یابی با نهانگاه دسته‌ها و جستجوی محدوده‌ای، اشیای صفحه، زبان مخصوص دامنه، کارخانه‌ی درایور با اتصال فازی، و جداسازی داده و گزارش‌گیری — پایداری و خوانایی تست‌ها را فراهم می‌کند. دو قرارداد پایداری (شناسه پایدار و قرارداد خوانش) رابطه‌ی برنامه‌ی هدف و چارچوب را قواعدمند کرده و تست نمودارهای گرافیکی را به تست داده‌ی خوانا تبدیل می‌کند.

برای ارزیابی، برنامه‌ی نمونه‌ی «ردیاب سیگنال ایران» (مبتنی بر Qt و SQLite) ساخته شد که جریان‌های دشوار تست شامل ورود چندپنجره‌ای، فیلترهای آبشاری، صفحه‌بندی، مجازی‌سازی جدول، نمودار و گزارش‌گیری را داراست. ۲۵ سناریوی سرتاسری روی آن پیاده شد که اجرای کاملشان پایدار، بازتولیدپذیر و در حدود شش دقیقه است و گزارش تعاملی Allure با پیوست تصویر خطا تولید می‌شود. مسیر انتقال چارچوب به پایپلاین‌ی CI[2]/CD[3] نیز طراحی شده است.

کلیدواژه‌ها: تست سرتاسری، خودکارسازی رابط کاربری، Qt، WinAppDriver، Appium، زبان مخصوص دامنه، شیء صفحه

[1] End-to-End
[2] Continuous Integration
[3] Continuous Delivery

فهرست مطالب

فصل ۱ مقدمه ۱

فصل ۲ مفاهیم پایه‌ای ۳

۱.۲ سطوح تست نرم‌افزار . . . ۳

۲.۲ خودکارسازی رابط کاربری در ویندوز . . . ۴

۳.۲ انتخاب ابزارهای زیرساخت و نقش هر یک . . . ۴

۱.۳.۲ WinAppDriver: دسترسی بیرونی به برنامه . . . ۴

۲.۳.۲ Appium: مدیریت پروتکل و جلسه‌ها . . . ۴

۳.۳.۲ Allure: گزارش‌گیری قابل‌فهم . . . ۵

۴.۳.۲ نقش هر یک در یک نگاه . . . ۵

۴.۲ تست سرتاسری وب در برابر برنامه‌های دسکتاپ . . . ۵

۵.۲ الگوی شیء صفحه و زبان مخصوص دامنه . . . ۶

۶.۲ فناوری‌های برنامه‌ی نمونه . . . ۶

۷.۲ pytest و گزارش‌گیری Allure . . . ۷

۸.۲ یکپارچه‌سازی و تحویل مستمر . . . ۷

فصل ۳ مرور کارهای مشابه ۸

۱.۳ ابزارهای تست خودکار وب . . . ۸

۲.۳ ابزارهای تجاری تست دسکتاپ . . . ۹

۳.۳ ابزارهای متن‌باز دسکتاپ و ویندوز . . . ۹

۴.۳ جمع‌بندی و جایگاه چارچوب حاضر . . . ۹

فصل ۴ معرفی برنامه‌ی نمونه ۱۰

۱.۴ کارکرد برنامه . . . ۱۰

۲.۴ داده‌های شبکه ۱۱
۳.۴ داشبورد و گزارش‌ها ۱۲
۴.۴ مجموعه‌داده ۱۴

فصل ۵ طراحی و پیاده‌سازی چارچوب **۱۵**
۱.۵ نگاه کلی به معماری ۱۵
۲.۵ قراردادهای پایداری در برنامه ۱۶
۳.۵ لایه‌ی مکان‌یابی و نهانگاه دسته‌ها ۱۷
۴.۵ کارخانه‌ی درایور و اتصال فازی ۱۷
۵.۵ ورودی صفحه‌کلید و اجرای قطعی ۱۷
۶.۵ تست با زبان دامنه در برابر کد خام درایور ۱۸
۷.۵ جداسازی پایگاه‌داده در سطح تست ۱۸
۸.۵ گزارش‌گیری و سوییت‌ها ۱۹
۹.۵ رفتارهای غیرمنتظره‌ی زیرساخت ۲۰

فصل ۶ ارزیابی و نتایج **۲۱**
۱.۶ محیط آزمایش ۲۱
۲.۶ سوییت سناریوها ۲۲
۳.۶ اجرای کامل و گزارش ۲۲
۴.۶ انتقال به پایپلاین CI/CD ۲۳
۵.۶ ارزش برای توسعه‌دهنده، تیم کیفیت و کاربر نهایی ۲۴
۶.۶ تحلیل نتایج ۲۴

فصل ۷ نتیجه‌گیری و کارهای آینده **۲۶**
۱.۷ نتیجه‌گیری ۲۶
۲.۷ کارهای آینده ۲۷

مراجع **۲۸**

واژه نامه انگلیسی به فارسی **۲۹**

واژه نامه فارسی به انگلیسی **۳۰**

فهرست شکل‌ها

۱.۴ دروازه‌ی ورود برنامه‌ی نمونه ۱۱
۲.۴ زبانه‌ی داده‌های شبکه؛ فیلتر استان فارس اعمال شده است ۱۲
۳.۴ داشبورد برنامه؛ نمودارهای دایره‌ای و ستونی سمت چپ و راست ۱۳
۴.۴ زبانه‌ی گزارش‌ها؛ خلاصه‌ی گروه‌بندی‌شده، نمودار دایره‌ای و روند روزانه ۱۴
۱.۵ معماری لایه‌ای چارچوب تست و ابزارهای جانبی آن ۱۶
۱.۶ گزارش Allure پس از اجرای کامل مجموعه؛ شکست عمدی سناریوی نمایشی آشکار است ۲۲
۲.۶ فیلم اجرای کامل سوییت سرتاسری (نیازمند نمایشگر PDF تعاملی) ۲۳

فصل ۱

مقدمه

نرم‌افزارهای دسکتاپ مبتنی بر Qt C++ سهم بزرگی از کاربردهای مهندسی، صنعتی و سازمانی را به خود اختصاص می‌دهند؛ از ابزارهای پایش شبکه گرفته تا نرم‌افزارهای کنترل تجهیزات. با این حال، بر خلاف برنامه‌های وب که اکوسیستم غنی‌ای از ابزارهای تست خودکار در اختیار دارند، تست خودکار رابط کاربری برای نرم‌افزارهای مبتنی بر Qt تا حد زیادی به ابزارهای تجاری محدود مانده است. نتیجه‌ی این خلأ معمولاً تست دستی، پوشش بازگشت ناکافی و هزینه‌ی بالای نگهداری کیفیت در هر تغییر است.

هدف این پایان‌نامه، طراحی و پیاده‌سازی یک E2E چارچوب تست سرتاسری برای نرم‌افزارهای مبتنی بر Qt C++ در بستر ویندوز است؛ چارچوبی که با ابزارهای متن‌باز ساخته شود، رفتار برنامه‌ی تحت تست را از دید کاربر واقعی بررسی کند و خروجی قابل‌فهمی برای تیم‌های توسعه و کیفیت تولید نماید. در این چارچوب، تعامل با برنامه از طریق خودکارسازی رابط کاربری[۱] انجام می‌شود: WinAppDriver درایور خودکارسازی ویندوز است و Appium به عنوان سرور پروتکل بین تست‌ها و درایور عمل می‌کند [۱، ۲].

میان‌بُرد اصلی این کار، ساختن یک DSL[2] زبان مخصوص دامنه در لایه‌ی بالای چارچوب است؛ به گونه‌ای که هر تست صرفاً با عباراتی مانند «واحد استان تهران را فیلتر کن، جستجو کن و تعداد ردیف‌ها را بخوان» نوشته شود و هیچ‌یک از تست‌ها مستقیماً با جزئیات درایور سروکار نداشته باشند. این جداسازی باعث می‌شود تغییرات رابط کاربری تنها در لایه‌ی اشیای صفحه شیء صفحه[۳] اعمال شود و سوییت‌ها دست‌نخورده بماند [۳، ۴].

[1] UI Automation
[2] Domain-Specific Language
[3] Page Object

برای ارزیابی چارچوب، یک برنامه‌ی نمونه با عنوان «ردیاب سیگنال ایران» طراحی و پیاده‌سازی شد؛ یک سامانه‌ی پایش کیفیت سیگنال مخابراتی مبتنی بر Qt Widgets و SQLite با داشبورد، نمودارها، فیلترها و گزارش‌گیری [۵، ۶]. این برنامه به عنوان SUT[4] بکار گرفته شده و مجموعه‌ای از ۲۵ سناریوی تست سرتاسری روی آن اجرا می‌شود که همه‌ی جریان‌های کاربر — از ورود و ثبت‌نام تا فیلترهای ترکیبی، صفحه‌بندی، نمودارها و گزارش‌گیری — را پوشش می‌دهند. خروجی هر اجرا در قالب گزارش Allure منتشر می‌شود [۷] و مسیر انتقال چارچوب به پایپلاین‌ی CD / CI نیز طراحی شده است.

ساختار ادامه‌ی پایان‌نامه چنین است: فصل ۲ مفاهیم پایه‌ای لازم را معرفی می‌کند؛ فصل ۳ مروری بر کارهای مشابه دارد؛ فصل ۴ برنامه‌ی نمونه را معرفی می‌کند؛ فصل ۵ طراحی و پیاده‌سازی چارچوب را تشریح می‌کند؛ فصل ۶ ارزیابی و نتایج را ارائه می‌دهد و فصل ۷ نتیجه‌گیری و کارهای آینده است.

[4] System Under Test

فصل ۲

مفاهیم پایه‌ای

هدف این فصل معرفی مبانی نظری و فناوری‌های مورد استفاده در چارچوب و برنامه‌ی نمونه است. نخست سطوح مختلف تست نرم‌افزار و جایگاه تست سرتاسری روشن می‌شود؛ سپس سازوکار خودکارسازی رابط کاربری در ویندوز، الگوی شیء صفحه و زبان مخصوص دامنه، و در پایان فناوری‌های به‌کاررفته در برنامه‌ی نمونه و ابزارهای گزارش‌گیری بررسی می‌شوند.

۱.۲ سطوح تست نرم‌افزار

تست نرم‌افزار معمولاً در چند سطح سازمان می‌یابد: تست واحد که رفتار کوچک‌ترین اجزا را بررسی می‌کند؛ تست یکپارچگی که درستی تعامل اجزا با یکدیگر را می‌سنجد و تست سرتاسری E2E که سامانه را از دید کاربر نهایی و به صورت یکپارچه تست می‌کند [۸]. هرچه از سطح واحد به سطح سرتاسری نزدیک‌تر می‌شویم، شبیه‌سازی واقعیت بیشتر می‌شود اما هزینه‌ی اجرا و نگهداری تست‌ها و شکنندگی آن‌ها در برابر تغییرات نیز افزایش می‌یابد. تست‌های سرتاسری کلاسیک به همین دلیل شکننده شناخته می‌شوند: کوچک‌ترین تغییر ظاهری رابط کاربری می‌تواند ده‌ها تست را از کار بیندازد. اصل طراحی چارچوب این پایان‌نامه، کاهش این شکنندگی از طریق قراردادهای پایداری و جداسازی لایه‌ها است که در فصل ۵ تشریح می‌شود.

۲.۲ خودکارسازی رابط کاربری در ویندوز

UIA[1] رابط برنامه‌نویسی دسترس‌پذیری ویندوز است که درخت سلسله‌مراتبی عناصر رابط کاربری هر برنامه را در اختیار می‌گذارد [۹]. هر عنصر ویژگی‌هایی مانند نام (Name) و شناسه‌ی خودکارسازی (AutomationId) دارد و الگوهای تعامل مانند فراخوانی دکمه یا تنظیم مقدار را ارائه می‌کند. برنامه‌های Qt این درخت را از روی ویژگی‌های objectName و accessibleName ساخت می‌دهند؛ این نکته پایه‌ی قرارداد «شناسه پایدار» در فصل ۵ است.

WinAppDriver [۲] درایور خودکارسازی مایکروسافت است که پروتکل W3C WebDriver را برای برنامه‌های ویندوز پیاده می‌کند و از طریق UIA با عناصر رابط کاربری کار می‌کند. Appium [۱] سرور پروتکل است که تست‌ها را از هر زبان برنامه‌نویسی به درایور مناسب ترجمه می‌کند. این ترکیب امکان می‌دهد تست‌ها با Python نوشته شوند و برنامه‌ی دسکتاپ دقیقاً مانند یک برنامه‌ی موبایل یا وب خودکار شود.

۳.۲ انتخاب ابزارهای زیرساخت و نقش هر یک

سه ابزار ستون فقرات چارچوب‌اند و هر یک بخش مشخصی از الزامات را پاسخ می‌دهد: دسترسی بیرونی به برنامه، مدیریت پروتکل و جلسه‌ها، و گزارش‌گیری قابل‌فهم. در ادامه سازوکار هر یک، دلیل انتخابش در برابر جایگزین‌ها و سهمش از معماری کلان توضیح داده می‌شود.

۱.۳.۲ WinAppDriver: دسترسی بیرونی به برنامه

WinAppDriver [۲] یک سرویس ویندوزی است که پروتکل استاندارد W3C WebDriver را برای برنامه‌های دسکتاپ پیاده می‌کند: دستورهای استاندارد (یافتن عنصر، کلیک، تایپ) را می‌گیرد و از طریق UIA [۹] به کنش واقعی در سیستم‌عامل ترجمه می‌کند. جلسه‌ی آن به یک پنجره‌ی سطح‌بالای مشخص گره می‌خورد (قابلیت appTopLevelWindow) و مختصات و رویدادها واقعی‌اند، نه شبیه‌سازی درون‌فرایندی. دلیل انتخاب در برابر جایگزین‌ها: رویکرد Squish [۱۵] برای دسترسی عمیق، کتابخانه‌ای در خود برنامه بارگذاری می‌کند که به معنای مجوز تجاری و از دست رفتن دید کاربر واقعی است؛ در مقابل WinAppDriver رسمی، رایگان و بدون هیچ تغییری در برنامه‌ی هدف کار می‌کند و چون همان پروتکل وب‌درایور را می‌گوید، ابزارهای اکوسیستم Appium مستقیماً روی آن سوار می‌شوند. سهم آن در الزامات: زیرساخت مکان‌یابی و تعامل بیرونی — همان سطحی که قرارداد شناسه پایدار روی آن سوار است.

۲.۳.۲ Appium: مدیریت پروتکل و جلسه‌ها

Appium [۱] یک سرور HTTP است که نقطه‌ی ورود استاندارد پروتکل وب‌درایور است: تست‌ها با کپابلیتی‌ها جلسه می‌سازند و سرور دستورها را به درایور مناسب (اینجا: درایور ویندوز، یعنی WinAppDriver) مسیریابی می‌کند. کتابخانه‌ی کلاینت Python آن همین پروتکل را به شیءهای ساده‌ی pytest ترجمه می‌کند و مدیریت مهلت‌ها، نشست‌ها و رفتار عنصرها را یک‌جا حل می‌کند.

[1] UI Automation

دلیل انتخاب: نوشتن مستقیم روی WinAppDriver یعنی پذیرفتن جزئیات خام پروتکل در همه‌ی تست‌ها؛ لایه‌ی Appium این جزئیات را می‌بلعد، کلاینت‌های زبان‌های مختلف ارائه می‌دهد و تعویض درایور (مثلاً مسیر اجرای بی‌سر با اتصال مستقیم به WinAppDriver) تنها یک تغییر پیکربندی است نه بازنویسی آزمون‌ها. سهم آن در الزامات: استقلال زبان و فریم‌ورک تست (pytest به عنوان میزبان)، مدیریت چرخه‌ی جلسه‌ها و امکان جابه‌جایی زیرساخت بدون دست زدن به لایه‌های بالا.

۳.۳.۲ Allure: گزارش‌گیری قابل‌فهم

Allure [۷] یک گزارش‌ساز تست است: افزونه‌ی pytest آن در حین اجرا، نتایج، گام‌ها و پیوست‌ها (مهم‌تر از همه تصویر لحظه‌ی خطا) را به صورت فایل‌های نتیجه‌ی ساخت‌یافته ثبت می‌کند و دستور allure generate از آن‌ها یک گزارش تعاملی HTML با دسته‌بندی شکست‌ها، جستجو، سابقه‌ی اجراها و زمان‌بندی هر تست می‌سازد که باید روی سرور محلی سرو شود. دلیل انتخاب: خروجی پیش‌فرض pytest متن خلاصه‌ی کنسول است که برای مرور مدیریتی و ثبت شکست‌ها قابل‌استفاده نیست؛ جایگزین‌های رایگان (فایل JUnit XML و نمای ساده‌ی آن در ابزارهای CI) فقط «گذشت یا نگشت» را می‌گویند، در حالی که گزارش Allure به هر شکست، تصویر لحظه‌ی وقوع، گام‌های طی‌شده و شناسه‌ی دقیق سناریو را پیوست می‌کند. سهم آن در الزامات: خروجی قابل‌فهم برای تیم کیفیت و جلسات دمو، پیوست خودکار تصویر خطا و فهرست‌بندی سناریوها به تفکیک سوییت.

۴.۳.۲ نقش هر یک در یک نگاه

جدول ۱.۲: سهم هر ابزار زیرساختی از الزامات چارچوب

Tool	سازوکار	بخش پاسخ‌داده‌شده از الزامات
WinAppDriver	پروتکل وب‌درایور روی UIA	دسترسی بیرونی بدون تغییر برنامه
Appium	سرور پروتکل و مدیریت جلسه	استقلال زبان تست و جابه‌جایی درایور
Allure	ثبت نتایج و پیوست‌ها، ساخت گزارش	گزارش قابل‌فهم و مستندسازی شکست

جریان کل به این شکل است: تستِ pytest از طریق کلاینتِ Appium دستور می‌دهد؛ Appium آن را به WinAppDriver می‌رساند؛ WinAppDriver از طریق UIA با برنامه اجرا می‌کند؛ و افزونه‌ی Allure در همان حین، گام‌ها و پیوست‌های هر تست را برای گزارش‌ساز ثبت می‌کند.

۴.۲ تست سرتاسری وب در برابر برنامه‌های دسکتاپ

مقایسه‌ی تست خودکارِ وب با تست خودکارِ دسکتاپ نشان می‌دهد چرا تست برنامه‌های Qt ذاتاً دشوارتر است. در وب، تست با یک مدل سند (DOM) کار می‌کند که همیشه کامل در دسترس است: هر عنصر شناسه‌ی پایدار (id) ، سلکتور CSS و متن جستجوپذیر دارد، صفحه در یک پنجره‌ی واحد زندگی می‌کند و اجرای بی‌سر با مرورگر بی‌سروصدا و ارزان است؛ ابزارهای مدرن وب حتی می‌توانند اسکریپت به صفحه تزریق کنند، وضعیت داخلی را بخوانند و رویدادها را مصنوعی صادر کنند [۱۴، ۱۲].

در دسکتاپ هیچ DOM ای وجود ندارد؛ تنها درخت دسترس‌پذیری است که اگر برنامه آن را درست بسازد (قرارداد شناسه

پایدار) خوانا و در غیر این صورت پر از عناصر بی‌نام است. جدول‌های داده‌ای بزرگ مجازی می‌شوند و تنها ردیف‌های قابل‌مشاهده در درخت حاضرند؛ منوها و پنجره‌های بازشو درخت را در همان لحظه‌ی تعامل تغییر می‌دهند و ورودی‌ها اغلب فقط از مسیر صفحه‌کلید واقعی پذیرفته می‌شود. پنجره‌ی ورود و پنجره‌ی اصلی دو درخت جدا هستند و جلسه‌ی درایور به یکی از آن‌ها گره می‌خورد. اجرای بی‌سر نیز به معنای «بستن چشم» نیست: ویندوز باید پنجره‌ی واقعی بسازد و پیش‌زمینه و مختصاتِ واقعی‌اند، نه مختصات سند. جمع این عوامل یعنی در وب هزینه‌ی خودکارسازی عمدتاً «نوشتن سلکتور» است، اما در دسکتاپ هزینه‌ی واقعی «مهندسی زیرساخت مکان‌یابی و تعامل» است؛ همان چیزی که لایه‌های چارچوب حاضر برای آن ساخته شده‌اند.

۵.۲ الگوی شیء صفحه و زبان مخصوص دامنه

در الگوی شیء صفحه POM[2] [۳] برای هر صفحه یا پنجره‌ی برنامه، یک کلاس مسئول ساخته می‌شود که تنها نهادِ آن صفحه را می‌شناسد و تعاملات را پنهان می‌کند. تست‌ها با شیء صفحه گفت‌وگو می‌کنند و از جزئیات مکان‌یابی عناصر بی‌خبرند. لایه‌ی بالاتر، زبان مخصوص دامنه DSL است: مجموعه‌ای از متدهای معنادار برای حوزه‌ی مسئله که خواندن تست‌ها را به روایت رفتار کاربر تبدیل می‌کند. ترکیب این دو لایه یک اصل ساده می‌سازد: «فقط لایه‌ی پایین با درایور حرف می‌زند». به این ترتیب تست‌ها مستند زنده‌ی رفتار برنامه‌اند و نگهداری آن‌ها محدود به یک لایه است.

روی آوردن جهانی به زبان‌های مخصوص دامنه در تست، تصادفی نیست و پیشینه‌ای روشن دارد: SQL کهن‌ترین و پذیرفته‌شده‌ترین زبان دامنه است که به جای فراخوانی‌های کتابخانه‌ای، پرسش را با واژگان خودِ داده بیان می‌کند؛ در خودکارسازی وب، سلکتورهای CSS یک زبان دامنه برای مکان‌یابی‌اند و Cucumber [۸] با زبان رفتارنویسی Gherkin تست را به متنی خوانا برای غیربرنامه‌نویس‌ها تبدیل می‌کند؛ Selenium برای هر زبان برنامه‌نویسی پیوندی می‌سازد، Cypress مفاهیم سفارشی برای برنامه‌های وب تعریف می‌کند و Playwright تست‌ها را با فیکسچرهای معنادار می‌نویسد. دلایل مشترک همه‌ی این‌ها یکسان است: واژگان دامنه خواندن و بازبینی تست را برای همه‌ی اعضای تیم (تست‌کننده، توسعه‌دهنده، مدیر محصول) ممکن می‌کند؛ دانش فنی زیرساخت در یک لایه متمرکز می‌شود و هزینه‌ی تغییر رابط کاربری به همان لایه محدود می‌ماند؛ تستِ خوانا، مستندِ زنده‌ی رفتار است و تازه‌واردان را بدون آموزش زیرساخت وارد تیم می‌کند. نوشتن تست با کد خام درایور، دقیقاً عکس همه‌ی این‌هاست: هر تست مستقل از بقیه دانش فنی را بازتولید می‌کند و هر تغییر ظاهری، ده‌ها نقطه را هم‌زمان می‌شکند. مقایسه‌ی عملی این دو رویکرد در فصل فصل ۵ آمده است.

۶.۲ فناوری‌های برنامه‌ی نمونه

برنامه‌ی نمونه با Qt Widgets نسخه‌ی ۸.۶ ساخته شده است [۵]؛ کتابخانه‌ی استاندارد رابط کاربری دسکتاپ Qt که درخت عناصر خود را برای UIA آشکار می‌کند. نمودارها با ماژول Qt Charts ترسیم می‌شوند و داده‌ها در پایگاه‌داده‌ی SQLite ذخیره می‌شوند [۶]. لایه‌ی دسترسی به داده‌ی برنامه به صورت یک کلاس واحد است که منبع یگانه‌ی حقیقت دامنه محسوب می‌شود و هم رابط کاربری و هم تست‌های یکپارچگی از آن تغذیه می‌کنند. علاوه بر تست‌های سرتاسری، برنامه‌ی نمونه دو تست داخلی دارد: تست

[2] Page Object Model

دود رابط کاربری و تست لایه‌ی داده که با Qt Test [۱۰] نوشته شده‌اند و در پایپلاین پیش از تست‌های سرتاسری اجرا می‌شوند.

۷.۲ pytest و گزارش‌گیری Allure

تست‌های چارچوب با فریم‌ورک pytest [۱۱] نوشته می‌شوند که از فیکسچرها داده‌ی پایه‌ی تست[۳] برای آماده‌سازی محیط هر تست بهره می‌گیرد. گزارش‌گیری با Allure [۷] انجام می‌شود؛ هر گام تست، فایل پیوست (از جمله تصویر لحظه‌ی خطا) و نتیجه‌ی هر تست در فایل‌های نتیجه‌ی Allure ثبت و سپس به گزارش تعاملی تبدیل می‌شود. این گزارش نقش مستندات اجرایی چارچوب را نیز ایفا می‌کند.

۸.۲ یکپارچه‌سازی و تحویل مستمر

در مهندسی نرم‌افزار به مجموعه‌ی اعمال یکپارچه‌سازی مداوم و تحویل پیوسته یا استقرار پیوسته، CI/CD می‌گویند؛ مجموعه‌ی رویه‌هایی که در آن‌ها هر تغییر کد به صورت خودکار ساخته، تست و گزارش می‌شود [۸]. در فصل ۶ شکل پایپلاین‌ای که چارچوب را به این چرخه منتقل می‌کند طراحی شده است؛ چارچوب از همان ابتدا با پیش‌نیازهای آن (بررسی سلامت ساختمان برنامه، برپایی خودکار سرور درایور و خروجی‌های قابل‌انتشار) طراحی شده است.

[3] Fixture

فصل ۳

مرور کارهای مشابه

برای جایگاه‌یابی چارچوب، خانواده‌های اصلی ابزارهای تست خودکار رابط کاربری مرور می‌شوند و تفاوت رویکرد این پایان‌نامه با آن‌ها بیان می‌گردد.

۱.۳ ابزارهای تست خودکار وب

Selenium [۱۲] کهن‌ترین و گسترده‌ترین استاندارد خودکارسازی مرورگر است که بر پایه‌ی پروتکل WebDriver کار می‌کند. Cypress [۱۳] و Playwright [۱۴] نسل جدیدترند که با اجرای تست در همان فرایند یا با کنترل مستقیم مرورگر، سرعت و پایداری بهتری ارائه می‌دهند. این ابزارها برای برنامه‌های وب ساخته شده‌اند و به درخت عناصر مرورگر دسترسی دارند؛ برای برنامه‌های دسکتاپ مبتنی بر Qt کاربرد مستقیمی ندارند. با این حال ایده‌های آن‌ها — پروتکل استاندارد درایور، جداسازی لایه‌ی مکان‌یابی از تست‌ها و گزارش‌گیری غنی — الهام‌بخش طراحی چارچوب حاضر بوده است.

۲.۳ ابزارهای تجاری تست دسکتاپ

برای دسکتاپ، ابزارهای تجاری متعددی وجود دارند؛ از جمله Ranorex و TestComplete که با بازشناسی اشیاء روی انواع فناوری‌های رابط کاربری کار می‌کنند. برای Qt به‌طور خاص، ابزار Squish [۱۵] شناخته‌شده‌ترین گزینه است که از طریق بارگذاری کتابخانه در خود برنامه، به اشیاء داخلی Qt دسترسی عمیق دارد. نقطه ضعف این خانواده هزینه‌ی مجوز، بسته‌بودن متن و وابستگی به فروشنده است؛ همچنین دسترسی عمیق به اشیاء داخلی، برنامه را از دید کاربر واقعی دور می‌کند و رفتار برنامه را در شرایط واقعی نمایش نمی‌دهد.

۳.۳ ابزارهای متن‌باز دسکتاپ و ویندوز

در سمت متن‌باز، WinAppDriver [۲] از طریق UIA [۹] با هر برنامه‌ی ویندوزی — بدون نیاز به تغییر در خود برنامه — کار می‌کند و پروتکل آن همان WebDriver است؛ اما به تنهایی فاقد لایه‌های معماری است: مکان‌یابی، نهان‌گاه دسته‌ها، مدیریت چرخه‌ی پنجره‌ها و گزارش‌گیری بر عهده‌ی نویسنده‌ی تست می‌ماند. Qt Test [۱۰] نیز چارچوب تست داخلی Qt است که در سطح واحد و یکپارچگی (و حتی تست رابط کاربری درون‌فرایندی) قوی است، اما برنامه را از بیرون و به شکل کاربر واقعی تست نمی‌کند.

۴.۳ جمع‌بندی و جایگاه چارچوب حاضر

جدول ۱.۳: مقایسه‌ی رویکردهای تست خودکار رابط کاربری

Tool	متن‌باز	دسترسی	دید تست
Selenium/Cypress/Playwright	بله	وب	مرورگر
Squish	خیر	اشیاء داخلی	درون‌فرایندی
Qt Test	بله	درون‌فرایندی	سطح واحد
WinAppDriver تنها	بله	UIA	بیرونی
چارچوب حاضر	بله	UIA + لایه‌ها	بیرونی، کاربر واقعی

چنان‌که در جدول جدول ۱.۳ دیده می‌شود، خلأ موجود، ترکیب سه چیز است: دسترسی بیرونی و بدون تغییر به برنامه، لایه‌های معماری (اشیاء صفحه، زبان مخصوص دامنه، جداسازی داده و گزارش‌گیری) و متن‌باز بودن کامل. چارچوب حاضر دقیقاً این ترکیب را هدف گرفته است: از WinAppDriver و Appium به عنوان زیرساخت استفاده می‌کند و آنچه که این زیرساخت نمی‌دهد — پایداری، خوانایی و نگهداشت‌پذیری تست‌ها — را با لایه‌های معماری و قراردادهای پایداری می‌سازد.

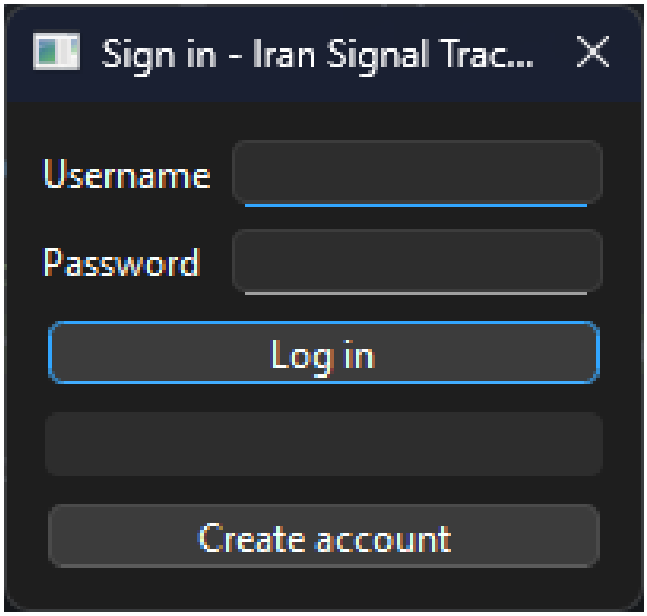
فصل ۴

معرفی برنامه‌ی نمونه

چارچوب تست برای ارزیابی، به یک SUT نیاز دارد؛ برنامه‌ای که به اندازه‌ی کافی پیچیده باشد تا توانایی چارچوب در تست «برنامه‌های پیچیده» را نشان دهد، اما همچنان در دامنه‌ی یک پروژه‌ی کارشناسی بماند. برای این منظور برنامه‌ی دسکتاپ «ردیاب سیگنال ایران» طراحی و پیاده‌سازی شد: سامانه‌ی پایش کیفیت سیگنال مخابراتی مبتنی بر Qt C++ و SQLite. این فصل صرفاً معرفی‌کننده‌ی برنامه است؛ موضوع اصلی پایان‌نامه چارچوب تست است که در فصل ۵ می‌آید.

۱.۴ کارکرد برنامه

برنامه از چهار بخش تشکیل شده است. نخست، دروازه‌ی ورود: کاربر باید با نام کاربری و گذرواژه وارد شود یا حساب جدید بسازد (شکل شکل ۱.۴)؛ حساب‌ها در همان پایگاه‌داده ذخیره می‌شوند و حساب مدیر امکان حذف کاربران را دارد. پس از ورود، پنجره‌ی اصلی با زبانه‌های «خانه»، «داده‌های شبکه»، «داشبورد» و «گزارش‌ها» در دسترس است.

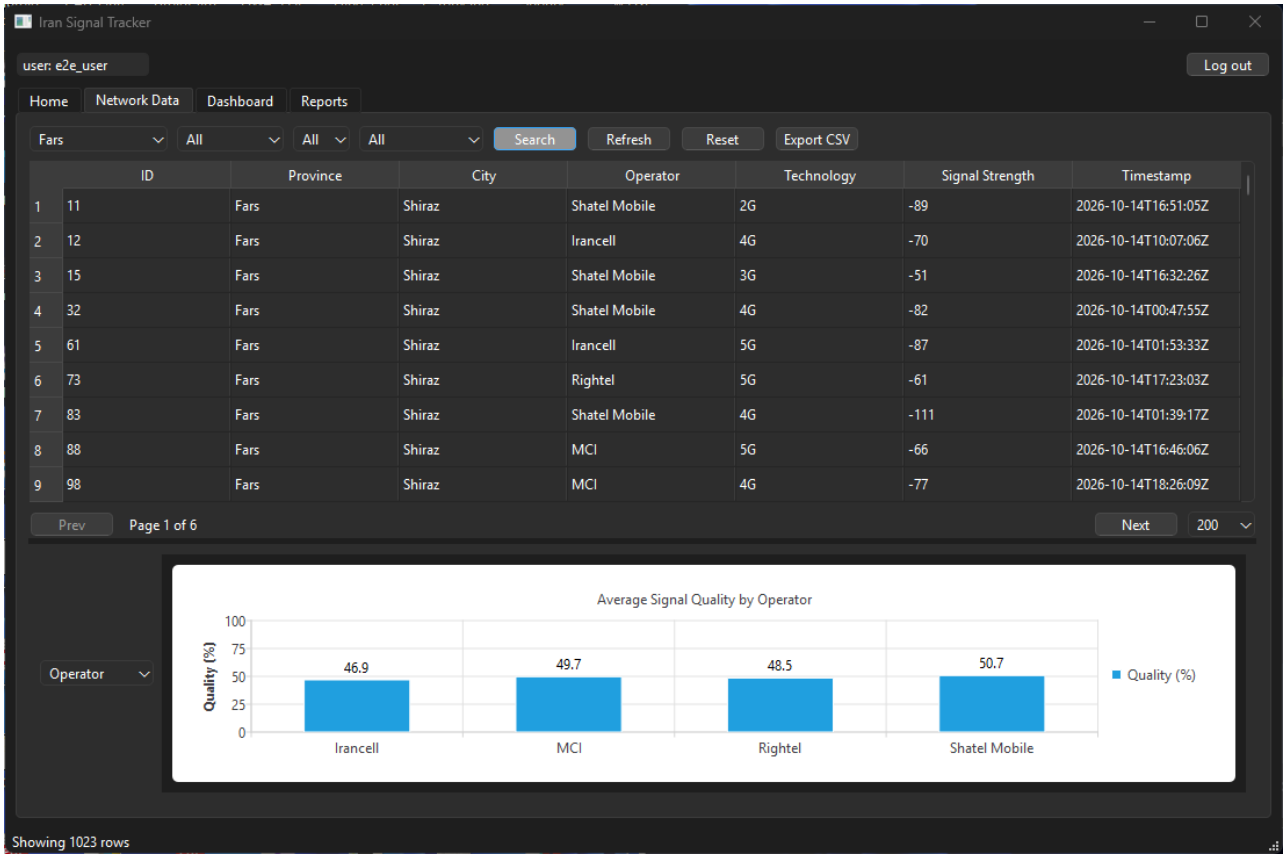


شکل ۱.۴: دروازه‌ی ورود برنامه‌ی نمونه

۲.۴ داده‌های شبکه

زبانه‌ی داده‌های شبکه (شکل شکل ۲.۴) هسته‌ی برنامه است: جدولی از نمونه‌های سیگنال با فیلترهای استان (با شهرهای آبشاری)، فناوری و اپراتور، جستجو، بازنشانی، خروجی CSV[1] و نمودار ستونی میانگین کیفیت. برای مجموعه‌داده‌های بزرگ، جدول صفحه‌بندی می‌شود (اندازه‌های صفحه ۵۰ تا ۱۰۰۰) و نوار وضعیت همیشه اندازه‌ی کامل نتیجه را نشان می‌دهد.

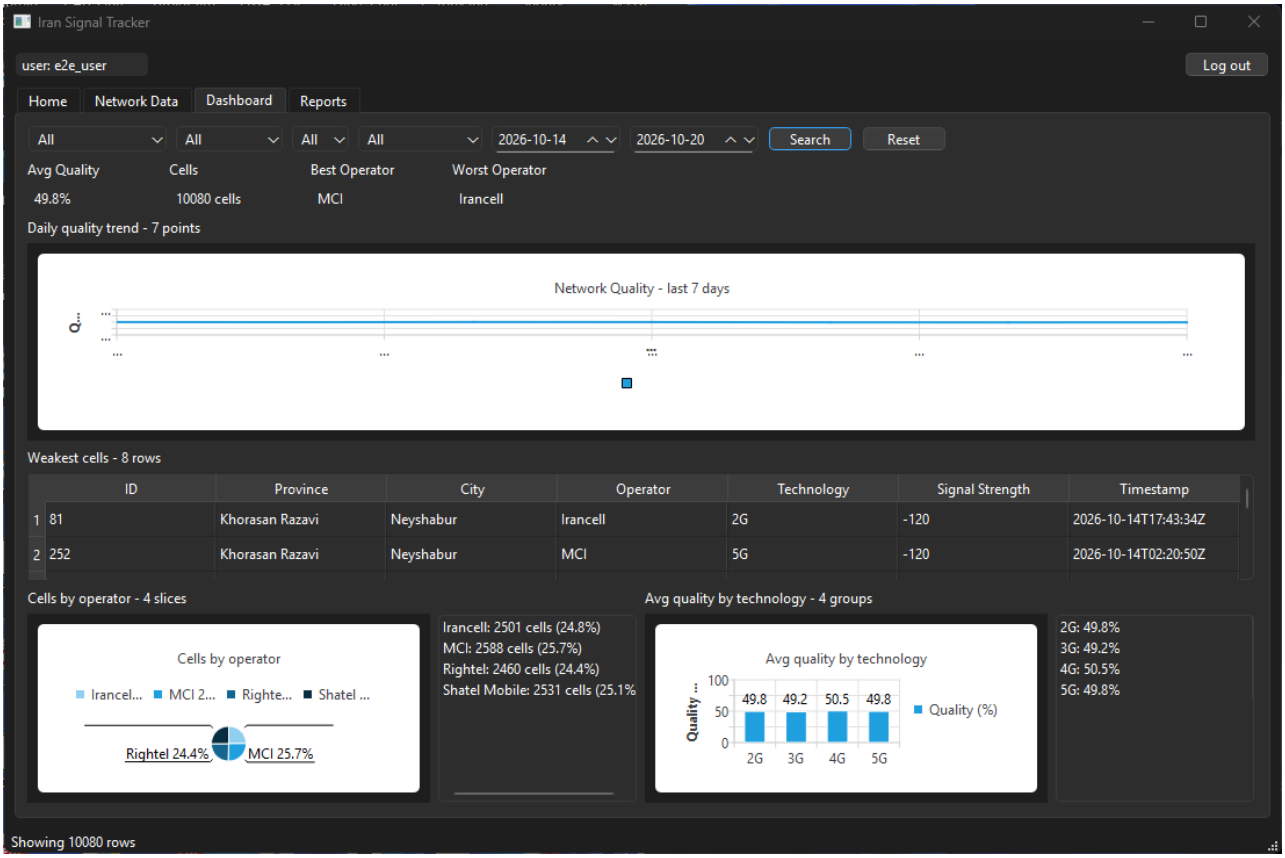
[1] Comma-Separated Values



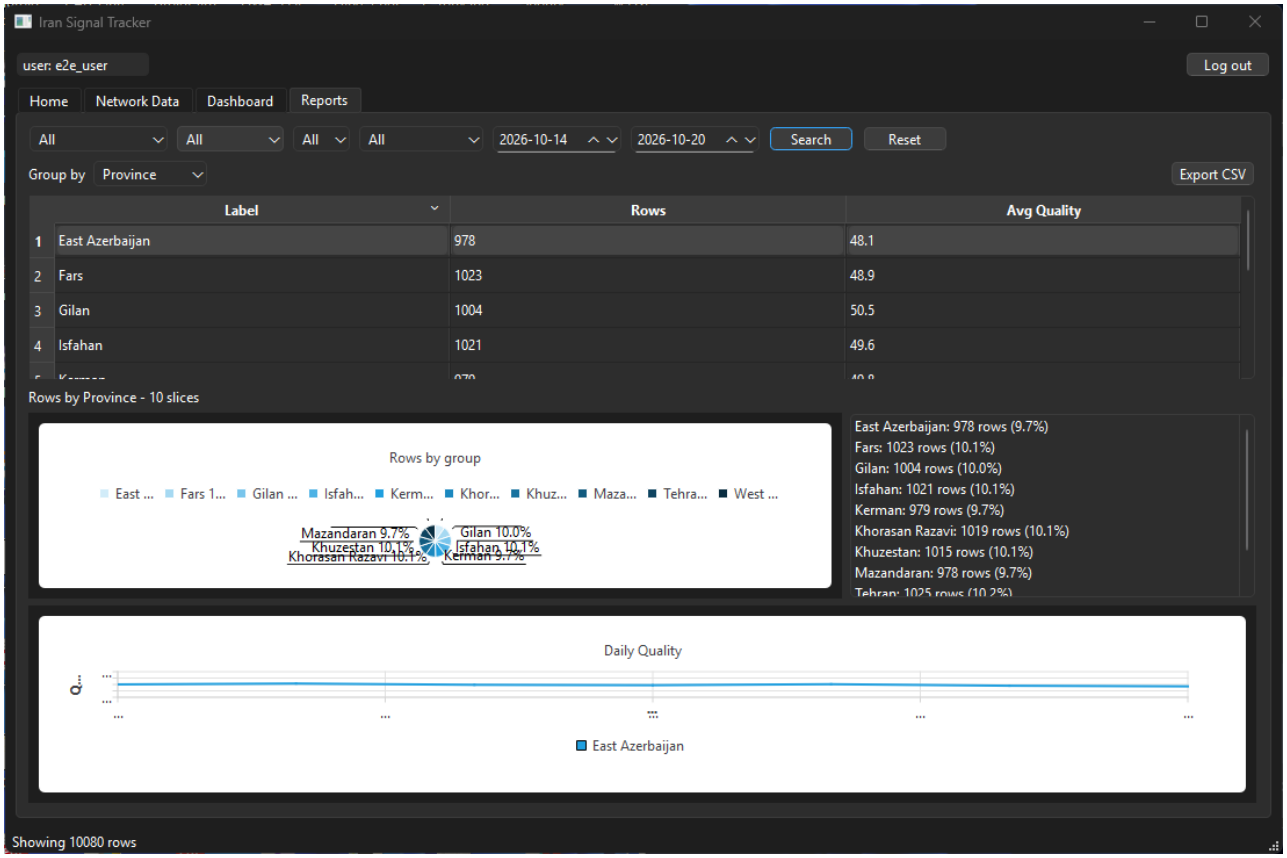
شکل ۲.۴: زبانه‌ی داده‌های شبکه؛ فیلتر استان فارس اعمال شده است

۳.۴ داشبورد و گزارش‌ها

داشبورد (شکل شکل ۳.۴) شاخص‌های کلیدی مانند میانگین کیفیت و تعداد نمونه‌ها، روند روزانه‌ی کیفیت، فهرست ضعیف‌ترین نمونه‌ها، نمودار دایره‌ای سهم اپراتورها و نمودار ستونی کیفیت بر حسب فناوری را نشان می‌دهد. هر دو نمودار جدید از فیلترهای همان زبانه تبعیت می‌کنند. زبانه‌ی گزارش‌ها (شکل شکل ۴.۴) جدول خلاصه‌ی گروه‌بندی‌شده (بر اساس استان، اپراتور یا فناوری)، نمودار دایره‌ای سهم هر گروه، نمودار روند روزانه‌ی گروه انتخاب‌شده و خروجی CSV را ارائه می‌دهد.



شکل ۳.۴: داشبورد برنامه؛ نمودارهای دایره‌ای و ستونی سمت چپ و راست



شکل ۴.۴: زبانه‌ی گزارش‌ها؛ خلاصه‌ی گروه‌بندی‌شده، نمودار دایره‌ای و روند روزانه

۴.۴ مجموعه‌داده

داده‌ی برنامه یک مجموعه‌ی شبه‌واقعی از کیفیت سیگنال مخابراتی است: ۱۰،۰۸۰ نمونه در بازه‌ی هفت روز (۱،۴۴۰ نمونه در روز)، با تولید قطعی مبتنی بر بذر ثابت تا هر اجرا بازتولیدپذیر باشد. داده‌ها شامل ده استان، شهرهای متناظر از نقشه‌ی دامنه‌ی برنامه، چهار اپراتور و چهار فناوری 2G تا 5G هستند. قدرت سیگنال هر نمونه به سنجه‌ی کیفیت ۰ تا ۱۰۰ درصد نگاشت می‌شود و همه‌ی نمودارها از این نگاشت مشترک تغذیه می‌کنند.

این برنامه آگاهانه ویژگی‌هایی را دارد که تست سرتاسری را دشوار می‌کند: جریان ورود با پنجره‌های متعدد، فیلترهای وابسته به هم، صفحه‌بندی، مجازی‌سازی جدول، نمودارهای گرافیکی که داده‌شان در درخت دسترس‌پذیری دیده نمی‌شود و دو نقش کاربری. همین دشواری‌ها معیار ارزیابی چارچوب در فصل ۶ هستند.

فصل ۵

طراحی و پیاده‌سازی چارچوب

این فصل هسته‌ی پایان‌نامه است: معماری لایه‌ای چارچوب، قراردادهای پایداری، لایه‌های مکان‌یابی، اشیای صفحه، زبان مخصوص دامنه، مدیریت درایور و پنجره‌ها، جداسازی داده و گزارش‌گیری. انگیزه‌ی هر تصمیم، یکی از دشواری‌های واقعی تست رابط کاربری است که در طول پیاده‌سازی مشاهده و مستند شد.

۱.۵ نگاه کلی به معماری

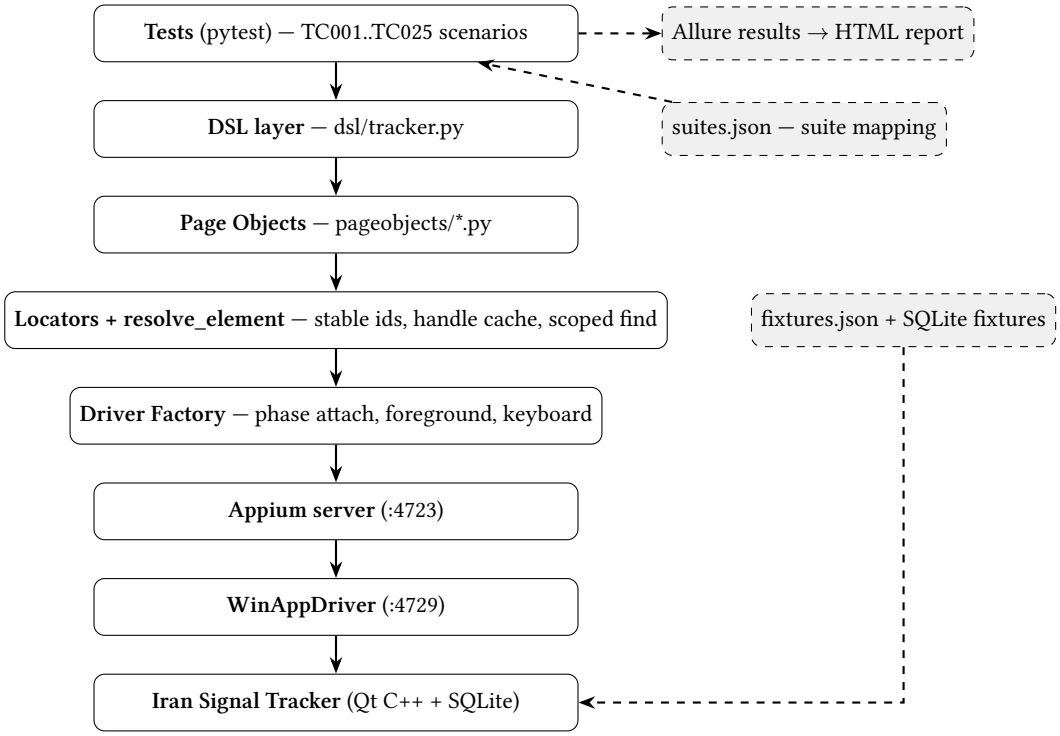
چارچوب از پنج لایه تشکیل شده که از بالا به پایین با هم صحبت می‌کنند و فقط لایه‌ی پایین‌تر با درایور سروکار دارد:

- **تست‌ها:** سناریوهای رفتاری؛ تنها از لایه‌ی زبان دامنه استفاده می‌کنند و هیچ فراخوانی درایوری ندارند.
- **لایه‌ی زبان دامنه (DSL):** پوشش معنادار برنامه («واحد استان را فیلتر کن»، «مجموع برش‌های نمودار دایره‌ای را بخوان»).
- **لایه‌ی اشیای صفحه (شیء صفحه):** برای هر صفحه/زبانه یک کلاس؛ تنها نهاد همان صفحه را می‌شناسد.
- **لایه‌ی مکان‌یابی:** مکان‌یابی کنترل‌ها با استراتژی‌های چندگانه، نهان‌گاه دسته‌ها نهانگاه دسته‌ها[1] و جستجوی محدوده‌ای.
- **لایه‌ی درایور:** کارخانه‌ی درایور[2]، برپایی سرور، اجبار پیش‌زمینه و ورودی صفحه‌کلید سطح سیستم‌عامل.

[1] Handle Cache

[2] Driver Factory

این جداسازی مستقیماً از الگوی شیء صفحه می‌آید [۳] اما با یک قاعده‌ی سخت‌گیرانه‌تر تکمیل شده است: دایرکتوری تست‌ها در بازرسی خودکار به فراخوانی‌های درایور حساس است و وجود حتی یک فراخوانی خام، ساخت را می‌شکند. به این ترتیب مرز لایه‌ها قرارداد شفاهی نمی‌ماند. شکل ۱.۵ این لایه‌بندی را به همراه ابزارهای جانبی (نگاشت سوییت‌ها، فیکسچرهای پایگاه‌داده و گزارش‌ساز) در کنار هم نشان می‌دهد: جریان اصلی از بالا به پایین است و هر لایه فقط با لایه‌ی مجاور خود صحبت می‌کند.



شکل ۱.۵: معماری لایه‌ای چارچوب تست و ابزارهای جانبی آن

برای اینکه خواننده بتواند در ادامه‌ی فصل مسیر هر جزء را دنبال کند، ساختار پوشه‌ی چارچوب چنین است (فایل‌های تکرارشونده خلاصه شده‌اند):

```
automation/
|-- config.py          # URLs, paths, seeded credentials
|-- conftest.py        # pytest fixtures: signed/admin/tracker + isolate_db
|-- fixtures.py        # DB restore + migrate engine (per-test isolation)
|-- locators.py        # stable-id locator candidates + readout ids
|-- suites.json        # suite name -> test files
|-- dsl/               # tracker.py (entry), network.py, dashboard.py,
|                      # reports.py, users.py, export_result.py
|-- drivers/           # factory.py (sessions), foreground.py, keyboard.py
|-- pageobjects/       # main_page.py, dashboard_view.py, reports_view.py,
|                      # resolve_element.py, filter_section.py, date_edit.py,
|                      # readout_list.py, tables, charts, signin, tabs
|-- tests/             # test_tc001..test_tc025 (25 scenarios) + fixtures unit
\-- tools/             # inspect_ui.py, probe_*.py, record_suite_video.py
```

دو مسیر خارج از این پوشه نیز در ادامه‌ی فصل نقش دارند: database/fixtures (فایل‌های پایگاه‌داده‌ی متعهدشده) و database/fixtures.json (نگاشت تست به فیکسچر) که در بخش جداسازی پایگاه‌داده می‌آیند، و اسکریپت‌های scripts/ (ساخت

برنامه، برپایی سرور درایور و اجرای سوییت‌ها) که در بخش پایپلاین به آن‌ها اشاره می‌شود.

۲.۵ قراردادهای پایداری در برنامه

شکننده‌ترین بخش تست‌های رابط کاربری، مکان‌یابی است. چارچوب دو قرارداد در سطح برنامه‌ی نمونه تعریف می‌کند که برنامه‌های هدف باید رعایت کنند:

۱. **شناسه پایدار**[۳]: هر کنترل تعاملی، نام دامنه‌ای ثابت خود را هم در objectName (که در UIA [۹] به AutomationId می‌رسد) و هم در accessibleName (که به Name می‌رسد) قرار می‌دهد. مکان‌یابی ابتدا با نام و در صورت نیاز با شناسه انجام می‌شود.

۲. **قرارداد خوانش**[۴]: عناصر فقط‌خواندنی (نشانگرهای کلیدی، برچسب‌های شمارنده) شناسه‌ی پایدار خود را در objectName نگه می‌دارند اما accessibleName آن‌ها همیشه متن نمایش‌داده‌شده را منعکس می‌کند؛ زیرا هیچ کلید ویژگی‌ای در WinAppDriver مقدار داخلی این کنترل‌ها را نمی‌خواند. نمودارهای گرافیکی نیز داده‌ی خود را در یک فهرست خوانا آینه می‌کنند تا تست‌ها به تصویر دسترسی نداشته باشند و به داده.

با این دو قرارداد، تستِ نمودار و نشانگر — که در درخت دسترس‌پذیری اصلاً دیده نمی‌شوند — به تست داده‌ی خوانا تبدیل می‌شود و تست‌ها در برابر تغییرات ظاهری مقاوم می‌مانند.

۳.۵ لایه‌ی مکان‌یابی و نهانگاه دسته‌ها

هر جستجوی ریشه‌ای در UIA کل درخت را بازمی‌گردد؛ در برنامه‌ای با جدول ده‌هزار ردیفی، هر جستجو تا بیش از ده ثانیه طول می‌کشد. سه سازوکار پایداری و سرعت پیاده شد: نخست، برنامه‌ی هدف بخش‌های کوچک نام‌گذاری‌شده (نوار فیلتر، نوار صفحه‌بندی) را در درخت آشکار می‌کند و لایه‌ی مکان‌یابی جستجو را به همان محدوده‌ی کوچک محدود می‌کند (پردازش اندازه‌گیری‌شده: از ۱۱ ثانیه به ۰/۰۲ ثانیه)؛ دوم، دسته‌ی (Handle) هر کنترل پس از نخستین یافتن در نهانگاه نهانگاه دسته‌ها با کلید (عنوان پنجره، شناسه کنترل) نگه داشته می‌شود و کار با دسته‌ی نهان‌شده در حد ۰/۰۱ ثانیه است؛ سوم، نهانگاه در هر تغییر پنجره باطل می‌شود (بخش اتصال فازی).

۴.۵ کارخانه‌ی درایور و اتصال فازی

WinAppDriver جلسه را به یک پنجره‌ی سطح‌بالا گره می‌زند؛ برنامه‌ی نمونه در جریان ورود، پنجره‌ی ورود را می‌بندد و پنجره‌ی اصلی می‌سازد و همین تغییر، جلسه‌ی قدیم را بی‌اعتبار می‌کند. کارخانه‌ی درایور فرایند برنامه را مالکیت می‌کند (اجرای برنامه، پایان دادن به آن در پایان جلسه) و جلسه را به عنوان پنجره‌ی سطح‌بالای برنامه می‌سازد؛ لایه‌ی زبان دامنه در هر فاز (ورود، پنجره اصلی) دوباره به پنجره‌ی جاری متصل می‌شود؛ به این سازوکار «اتصال فازی» اتصال فازی[۵] گفته می‌شود. علاوه بر این، دو رفتار

[3] Stable Identifier
[4] Readout Contract
[5] Phase Attach

خطرناک ویندوز مهار شده است: اول اینکه ورودی صفحه‌کلید به پنجره‌ی پیش‌زمینه می‌رسد و اگر برنامه در پیش‌زمینه نباشد، متن تست در پنجره‌ی دیگری (مثلاً یک پنجره گفتگو) تایپ می‌شود؛ کارخانه پیش از هر تعامل، پیش‌زمینه را به‌صورت قطعی به برنامه منتقل می‌کند. دوم اینکه send_keys درایور به متن اضافه می‌کند نه جایگزین؛ اشیای صفحه ورودی‌های متنی را پیش از تایپ خالی می‌کنند.

۵.۵ ورودی صفحه‌کلید و اجرای قطعی

تعامل با چرخاننده‌ها (کمبوها) از مسیر صفحه‌کلید انجام می‌شود: باز کردن پنجره‌ی بازشو با کلیک، تایپ سطح سیستم‌عامل متن گزینه (به‌کارافتادن جستجوی نوع‌نویسی Qt) و تأیید با Enter. این مسیر نه به مکان‌یابی عناصر پنجره‌ی بازشو نیاز دارد و نه در برابر هم‌نامی گزینه‌ها با سلول‌های جدول می‌شکند. همین الگو برای فیلترهای تاریخ تعمیم داده شد: تاریخ با ارسال هشت رقم از طریق خود درایور روی عنصر تاریخ تنظیم می‌شود و بخش‌های سال، ماه و روز به‌ترتیب پر می‌شوند. هر یک از این روش‌ها با یک تست کاوشی (probe) به صورت مستقل کشف و در توضیحات کد مستند شده است.

۶.۵ تست با زبان دامنه در برابر کد خام درایور

برای نشان‌دادن ارزش لایه‌ی زبان دامنه، همان سناریو را در دو سطح می‌بینیم. کد زیر همان تعامل با فراخوانی‌های خام درایور است؛ یعنی آنچه نویسنده‌ی تست بدون چارچوب می‌نویسد:

```
# raw driver access - what a tester writes WITHOUT the framework
driver = attach("Iran Signal Tracker")
section = driver.find_element(
    "xpath", "//*[@AutomationId='dashboardFilterSection']")
combo = section.find_element(
    "xpath", ".//*[@AutomationId='dashboardProvinceCombo']")
combo.click(); sleep(0.5)      # popup takes keyboard focus
keyboard.type_text("Tehran"); keyboard.press_enter()
sleep(0.6)                     # cascade settle
driver.find_element(
    "xpath", "//*[@Name='dashboardSearchButton']").click()
kpi = driver.find_element(
    "xpath", "//Edit[contains(@AutomationId,'kpiCells')]")
assert kpi.get_attribute("Name") == "1023 cells"
```

و این همان تست با زبان دامنه‌ی چارچوب:

```
dash = signed.dashboard()
dash.province("Tehran").search()
assert dash.kpi("cells") == "1023 cells"
```

تفاوت فقط کوتاه‌تر شدن کد نیست. در نسخه‌ی خام، نویسنده‌ی تست باید بداند مکان‌یابی با کدام استراتژی کار می‌کند، چرا بین کلیک و تایپ مکث لازم است، چرا ورودی باید از مسیر صفحه‌کلید سیستم‌عامل برود، و ده‌ها رفتار ریز زیرساخت که در جدول

جدول ۱.۵ فهرست شده‌اند؛ هر خطای ریز در این جزئیات، شکست نمایشیِ تست است نه شکست برنامه. در نسخه‌ی زبان دامنه، همه‌ی آن دانش یک بار در لایه‌ی اشیای صفحه زیست می‌کند و نویسنده‌ی تست با واژگان دامنه («استان»، «جستجو»، «خواندن نشانگر») کار می‌کند؛ اگر فردا جستجو از مسیر دیگری انجام شود، فقط یک کلاس تغییر می‌کند و هیچ تستی بازنویسی نمی‌شود. به بیان دیگر، نسخه‌ی خام دانش فنی را در همه‌جا پخش می‌کند و نسخه‌ی زبان دامنه آن را یک‌جا متمرکز می‌کند؛ همین تمرکز است که نگهداشت مجموعه‌ی ۲۵ سناریویی را برای یک نفر ممکن می‌کند.

۷.۵ جداسازی پایگاه‌داده در سطح تست

تست‌ها باید مستقل و بازتولیدپذیر باشند [۴]. هر تست پیش از اجرا، فایل پایگاه‌داده‌ی خود را از یک «فیکسچر ثابت» داده‌ی پایه‌ی تست از پیش متعهدشده بازمی‌گرداند و سپس مهاجرت‌های طرح را روی آن اعمال می‌کند. نگاشت تست به فیکسچر در یک فایل پیکربندی است و بازگردانی بی‌قیدوشرط است (نه «فقط اگر تغییر کرده») تا هیچ تستی به حالت جلسه‌ی قبلی وابسته نماند. خروجی‌های CSV نیز در هر تست پاک می‌شوند تا تست‌های گزارش‌گیری به فایل‌های باقی‌مانده از اجرای دیگر تکیه نکنند.

طراحی فیکسچرها عمداً چندپایگاه‌داده‌ای است: هر فیکسچر یک فایل SQLite متعهدشده با محتوای قطعی است و فایل پیکربندی JSON تعیین می‌کند هر تست کدام فیکسچر را دریافت کند؛ در حالت پیش‌فرض همه‌ی تست‌ها فیکسچر کامل با ۱۰٬۰۸۰ نمونه می‌گیرند و هر تست می‌تواند با یک نگاشت صریح فیکسچر خود را بازنویسی کند. چارچوب هم‌اکنون دو فیکسچر دارد: کامل و خالی (برای رفتار برنامه با داده‌ی صفر)، و افزودن فیکسچر تازه — مثلاً داده‌ی یک استان خاص برای تست‌های آینده — تنها یک فایل db. و یک خط در فایل پیکربندی است؛ هیچ تغییری در کد چارچوب لازم نیست. فایل نگاشت چنین است:

```
{
  "default": "seed_full",
  "tests/test_tc018_empty_fixture.py::test_tc018_empty_fixture":
    "empty"
}
```

کلید نخست، فیکسچر پیش‌فرض همه‌ی تست‌ها را تعیین می‌کند و هر کلید دیگر یک پوشش سرتاسری است: شناسه‌ی کامل تست (مسیر فایل + نام تابع) به فیکسچر موردنظر نگاشت می‌شود. این شکست «پیکربندی به جای کد» باعث می‌شود انتخاب داده‌ی هر تست در یک نگاه دیده و تغییرش بدون دست زدن به آزمون انجام شود. فیکسچرها هرگز در محل اجرا باز نمی‌شوند (فقط بازنویسی بایت‌به‌بایت) تا بایت‌های فایل متعهدشده سالم بمانند و نتیجه‌ی هر اجرا دقیقاً بازتولید شود.

۸.۵ گزارش‌گیری و سوییت‌ها

تست‌ها در فایل‌های مستقل به نام سناریو (از TC001 تا TC025) نوشته شده‌اند و فایل پیکربندی سوییت‌ها، نام هر سوییت (smoke، auth، filters، reporting، dashboard، data، unit) را به فهرست فایل‌های تست آن نگاشت می‌کند. اسکریپت اجرا پیش از هر چیز سلامت ساختمان برنامه و برپابودن سرور درایور را بررسی می‌کند، مجموعه را اجرا می‌کند و در پایان گزارش Allure [۷] را بسازد و روی یک سرور محلی باز کند؛ زیرا باز کردن گزارش از مسیر فایل، نسخه‌های جدید گزارش را می‌شکند.

تصویر لحظه‌ی خطا نیز به صورت خودکار به گزارش هر تست شکست‌خورده پیوست می‌شود. فایل نگاشت سوییت‌ها نمونه‌ای واقعی از همین پیکربندی است:

```
{
  "smoke":  ["test_tc001_launch", "test_tc012_login",
             "test_tc002_province_filter", "test_tc009_export_csv"],
  "auth":   ["test_tc011_register", "test_tc012_login",
             "test_tc013_logout", "test_tc015_users_admin"],
  "filters": ["test_tc002_province_filter",
              "test_tc003_city_filter", "..."],
  "reporting": ["test_tc009_export_csv",
                "test_tc010_chart_visibility", "..."],
  "dashboard": ["test_tc020_dashboard_filters",
                "test_tc021_dashboard_pie_vs_csv",
                "test_tc022_dashboard_tech_bar"],
  "data":  ["test_tc008_refresh", "test_tc016_pagination",
            "test_tc019_demo_failure", "..."],
  "unit":  ["test_fixtures_unit"]
}
```

هر نام سوییت به فهرست فایل‌های تست خود نگاشت می‌شود (پسوند py. اختیاری است) و اسکریپت اجرا با یک نام، آن سوییت را اجرا می‌کند؛ انتخاب سوییت در هنگام اجرا (مثلاً فقط auth در هر تغییر پوشش احراز هویت) بدون هیچ تغییری در کد ممکن است. این فایل در عمل نقش فهرست وام‌های کیفیت را دارد: هر سناریو دقیقاً در یک سوییت ثبت است و جابه‌جایی آن بین سوییت‌ها یک خط تغییر است.

۹.۵ رفتارهای غیرمنتظره‌ی زیرساخت

در طول پیاده‌سازی، رفتارهای غیرمنتظره‌ای از زیرساخت مشاهده و هر یک با تست کاوشی مستند شد. جدول ۱.۵ خلاصه‌ی آن‌هاست؛ ارزش این جدول در این است که هر یک به یک سازوکار دائمی در چارچوب تبدیل شده است.

جدول ۱.۵: رفتارهای غیرمنتظره‌ی مشاهده‌شده در زیرساخت و پاسخ چارچوب

رفتار مشاهده‌شده	پاسخ چارچوب
send_keys به متن اضافه می‌کند	پاک‌سازی پیش از تایپ در اشیای صفحه
جلسه به یک پنجره گره می‌خورد	اتصال فازی + باطل‌سازی نهانگاه
کلیک به پنجره‌ی غیرپیش‌زمینه می‌رسد	اجبار قطعی پیش‌زمینه
جستجوی ریشه‌ای بسیار کند	بخش‌های نام‌گذاری‌شده + نهانگاه دسته‌ها
عناصر جدول مجازی می‌شوند	شمارش از برچسب‌ها نه از ردیف‌ها
داده‌ی نمودار در درخت دیده نمی‌شود	قرارداد خوانش و فهرست آینه

نمونه‌ی زیر تستی است که تنها با زبان دامنه نوشته شده؛ خوانایی و نبودِ هر فراخوانی درایوری معیار طراحی این لایه است:

```
def test_tc021_dashboard_pie_vs_csv(signed):
    net = signed.network()
    net.province("Tehran")
    result = net.export().should_create_file()
    per_operator = count_operators(result.newest_file())
```

```
    dash = signed.dashboard()
    dash.province("Tehran").search()
    slices = dash.pie_slices()
    assert {label: count for label, count, _ in slices} \
        == dict(per_operator)
```

فصل ۶

ارزیابی و نتایج

در این فصل چارچوب بر معیارهای عملی ارزیابی می‌شود: پوشش سناریوها، نتیجه‌ی اجرای کامل، گزارش‌گیری و مسیر انتقال به پایپلاین‌ی CD/CI.

۱.۶ محیط آزمایش

همه‌ی ارزیابی روی یک سامانه‌ی ویندوزی واقعی انجام شده است؛ مشخصات محیط به پیروی از رویه‌ی گزارش‌های مهندسی، فهرست‌وار آورده می‌شود:

- سیستم‌عامل: ویندوز ۱۱ (میزکار) با فعال‌بودن حالت توسعه‌دهنده
- زبان و کیت برنامه‌ی نمونه: Qt 6.8.3 (MinGW 13.1), CMake, Ninja
- زبان چارچوب: Python 3.12 با pytest و Appium Python Client
- سرور درایور: Appium 3 روی درگاه ۴۷۲۳ و WinAppDriver برای مسیر اجرای بی‌سر[1]
- گزارش‌ساز: Allure 3
- سخت‌افزار: رایانه‌ی شخصی با پردازنده‌ی x86_64 و ۱۶ گیگابایت حافظه

[1] Headless Run

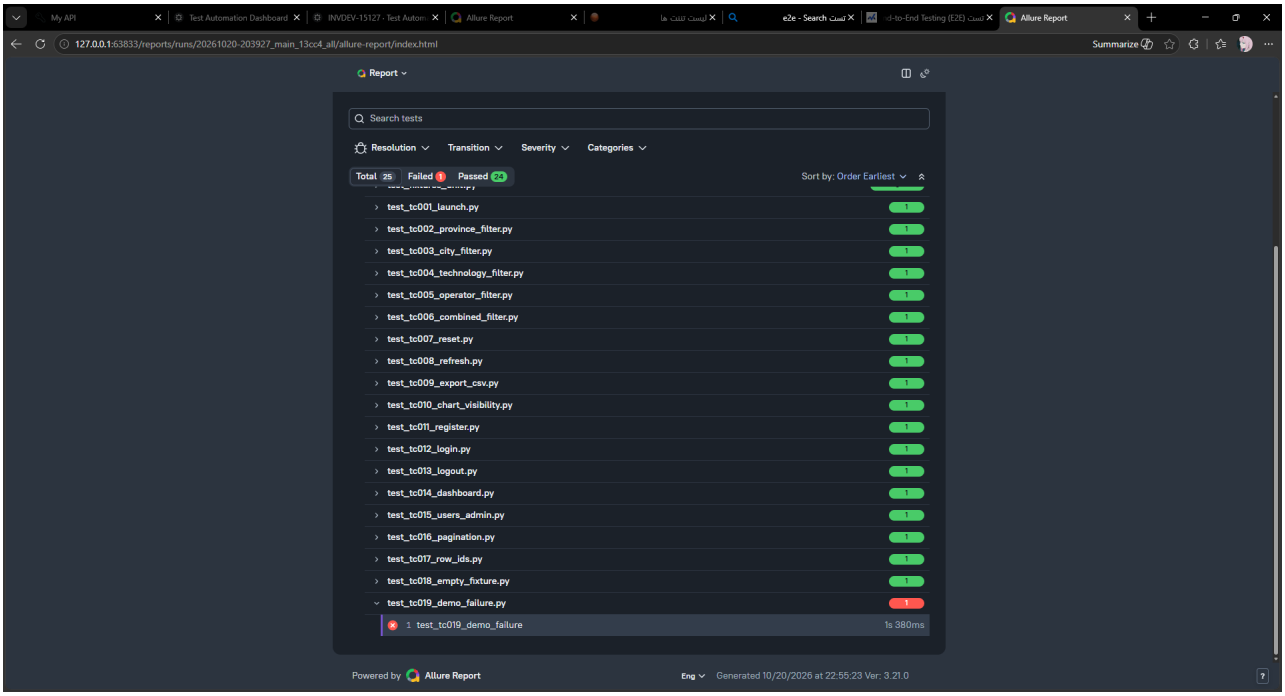
۲.۶ سوییت سناریوها

۲۵ سناریوی سرتاسری نوشته شده که به شش خانواده تقسیم می‌شوند: جریان ورود و ثبت‌نام (۵ سناریو)، فیلترهای جدول شبکه (۶ سناریو)، صفحه‌بندی و شناسه‌ی ردیف‌ها (۳ سناریو)، داشبورد (۴ سناریو شامل سنجش نمودار دایره‌ای و ستونی در برابر داده‌ی مستخرج از خروجی CSV)، گزارش‌ها (۴ سناریو شامل انتقال فیلتر و گروه‌بندی) و تست‌های زیرساخت (۲ سناریو). در کنار این‌ها، دو تست داخلی برنامه (تست دود رابط کاربری و تست لایه‌ی داده) و یک سناریوی نمایشی عمداً شکست‌خورده وجود دارد که نقش آن، نشان‌دادن رفتار گزارش‌گیری در هنگام خطا در جلسه‌ی نمایش است.

نکته‌ی مهم در سناریوهای نمودار این است که تست، خروجی CSV همان فیلتر را از زبانه‌ی شبکه استخراج و توزیع داده را مستقل از برنامه محاسبه می‌کند؛ سپس برش‌های خوانده‌شده از فهرست آینه‌ی نمودار را با آن مقایسه می‌کند. به این ترتیب درستیِ نمودار با «منبع حقیقت دیگری» سنجیده می‌شود نه با خود برنامه.

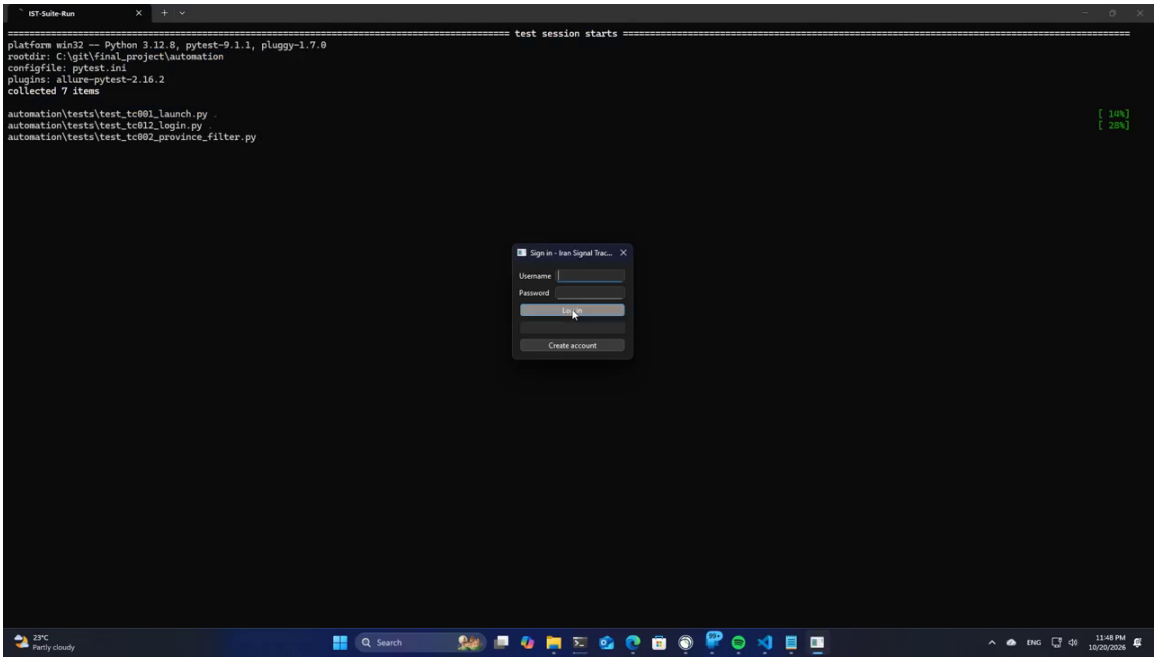
۳.۶ اجرای کامل و گزارش

اجرای کامل مجموعه در حدود شش دقیقه انجام می‌شود؛ خروجی پایدار آن ۲۴ سناریوی موفق و یک شکست عمدی است. گزارش Allure این اجرا در شکل ۱.۶ نمایش داده شده است: شمار کل، موفق و شکست به تفکیک فایل سناریو، با زمان‌بندی هر تست. در صورت شکست هر تست، تصویر لحظه‌ی خطا به گزارش پیوست می‌شود.



شکل ۱.۶: گزارش Allure پس از اجرای کامل مجموعه؛ شکست عمدی سناریوی نمایشی آشکار است

برای نشان‌دادن جریان کامل اجرا، یک فیلم کوتاه از اجرای کل مجموعه ضبط شده است. در نسخه‌ی الکترونیکی این پایان‌نامه، ویدیوی زیر با کلیک فعال می‌شود:



شکل ۲.۶: فیلم اجرای کامل سوییت سرتاسری (نیازمند نمایشگر PDF تعاملی)

در نسخه‌ی چاپی، شکل‌های شکل ۱.۴ تا شکل ۱.۶ در فصل‌های قبلی جریان را بازگو می‌کنند.

۴.۶ انتقال به پایپلاین CI/CD

چارچوب از ابتدا برای سوارشدن بر پایپلاین طراحی شده است. شکل هدف پایپلاین چنین است:

```
Commit
  -> Build App         (cmake configure + build; ctest smoke + db_test gate)
  -> Launch Appium     (appium server on :4723)
  -> Launch Application (driver factory starts the exe via caps)
  -> Execute Tests     (python -m pytest automation/tests)
  -> Generate Allure   (allure generate ... -o reports/allure-report)
  -> Publish Report    (host allure-report artefact, if: always())
```

نکات طراحی‌شده برای نویسنده‌ی پایپلاین: نهان‌گاه کردن مسیرهای Qt ، MinGW و Python بر اساس پین‌های نسخه در مستندات محیط؛ نیاز رانر به ویندوز، حالت توسعه‌دهنده و نصب WinAppDriver؛ شکست هر تست به معنی شکست کار بدون حذف انتشار گزارش (با شرط (always() ؛ و مسیر اجرای بی‌سر با DRIVER_MODE=direct که سرور Appium را از معادله حذف می‌کند و مستقیماً به WinAppDriver وصل می‌شود. جداسازی پایگاه‌داده در سطح تست (فصل ۵) ضمانت می‌کند که اجراهای موازی‌نما و متوالی روی یک رانر، به هم آلودگی ایجاد نکنند.

چگونگی کار تست در پایپلاین نیز قواعدی دارد که چارچوب به آن‌ها پاسخ می‌دهد: نخست، هر تغییر کد (هر commit یا درخواست ادغام) باید ماشه‌ی اجرا باشد تا بازخورد پیش از ادغام برسد؛ دوم، ترتیب مراحل ارزان به گران است — اول ساخت برنامه و تست‌های داخلی سریع (آزمون دود و لایه‌ی داده) به عنوان دریچه، سپس تست‌های سرتاسری — تا هزینه فقط روی کدهای

سالم پرداخت شود؛ سوم، رانر باید تازه و ایزوله باشد (حذف فرایندهای باقی‌مانده، بازگردانی فیکسچر پیش از هر تست) و نهان‌گاه مسیرهای سنگین Qt و MinGW هزینه‌ی راه‌اندازی را می‌کاهد؛ چهارم، خروجی هر اجرا باید به عنوان سازه‌ی قابل‌انتشار (گزارش Allure و نتایج آن) نگهداری شود تا شکست‌ها قابل مرور و مقایسه باشند؛ و پنجم، شکست هر تست باید کار را قرمز کند اما گزارش را هرگز حذف نکند. برای کوتاه‌شدن حلقه‌ی بازخورد، سوییت سریع‌تر (مثلاً smoke) را می‌توان روی هر تغییر اجرا کرد و سوییت کامل را پس از ادغام.

چارچوب به صورت متن‌باز در مخزن GitHub به نشانی https://github.com/AlirezaBKHN/Qtcpp_E2E_Testing در دسترس است. برای استفاده محلی پس از نصب پیش‌نیازها (مستندشده در مخزن): ساختن برنامه با scripts\build.ps1، برپایی سرور درایور با scripts\run-appium.ps1 و اجرای سوییت‌ها با scripts\run-tests.ps1؛ مسیر اجرای بی‌سر نیز تنها نیازمند نصب WinAppDriver و تنظیم متغیر DRIVER_MODE=direct است.

۵.۶ ارزش برای توسعه‌دهنده، تیم کیفیت و کاربر نهایی

در جریان‌های دستی، پیش از هر انتشار باید یک نفر تمام جریان‌های برنامه را به‌صورت دستی مرور کند: ورود، هر فیلتر به‌تنهایی و ترکیبی، صفحه‌بندی، نمودارها و گزارش‌ها؛ کاری چندساعته که در هر تغییر تکرار می‌شود، خطاپذیر است و با بزرگ‌شدن برنامه بزرگ‌تر می‌شود. در چارچوب حاضر، همان مرور به یک اجرای شش‌دقیقه‌ای تبدیل شده که ۲۵ سناریو را با نتیجه‌ی دقیق و گزارش مصور اجرا می‌کند؛ صرفه‌جویی مستقیمِ زمانِ تیم کیفیت در هر چرخه انتشار است و اندازه‌گیری‌شدنی: نسبت زمانِ مرور دستی به زمان اجرای خودکار با بزرگ‌شدن مجموعه به نفع خودکار تغییر می‌کند، چون افزودن یک سناریوی جدید هزینه‌ی یک‌باره دارد و اجرایش تقریباً رایگان است.

برای توسعه‌دهنده، ارزش در کوتاهی حلقه‌ی بازخورد است: به‌جای کشف یک پس‌رفت در مرحله‌ی انتشار (جایی که ردیابی گران‌ترین است)، اجرای سوییت روی هر تغییر، شکست را دقیقاً روی همان تغییر نشان می‌دهد با تصویر لحظه‌ی خطا و گزارش قابل جستجو. برای کاربر نهایی نیز اثر غیرمستقیم اما واقعی است: انتشارهای پرتکرارتر با اعتماد بیشتر، پس‌رفت‌های کمتری که به دست او برسند و رفع سریع‌تر خطاهای گزارش‌شده، چون تیم به‌جای صرف وقت در بازآزمایی دستی، روی ریشه‌یابی و رفع خطا تمرکز می‌کند. به بیان دیگر، تست سرتاسری خودکار، زمان تیم کیفیت را از «تکرار مراحل شناخته‌شده» به «کشف ناشناخته‌ها» منتقل می‌کند؛ جایی که قضاوت انسانی واقعاً ارزش دارد.

۶.۶ تحلیل نتایج

سه شاهد کیفی از ارزیابی قابل استناد است. نخست، پایداری: کل مجموعه در چندین اجرای پیاپی با همان نتیجه اجرا شده و تنها شکست‌ها، شکست عمدی سناریوی نمایشی بوده است. دوم، سرعت: با بخش‌بندی درخت مکان‌یابی و نهانگاه دسته‌ها، زمان اجرای کامل از حدی که عملاً غیرقابل‌استفاده بود (جستجوهای تک‌ثانیه‌ای تا ۱۱ ثانیه) به حدود شش دقیقه برای ۲۵ سناریو رسید. سوم، نگهداشت‌پذیری: در طول توسعه، تغییرات متعددی در چیدمان رابط کاربری برنامه (جابه‌جایی پنل‌ها، افزودن نمودار) رخ داد که هیچ‌یک تستی را نشکست؛ تنها لایه‌ی اشیای صفحه تغییر کرد — همان چیزی که جداسازی لایه‌ها وعده می‌داد. محدودیت‌های

چارچوب نیز روشن است: وابستگی به ویندوز و WinAppDriver، نیاز برنامه‌های هدف به رعایت قراردادهای پایداری، و اجرای تعاملی (نه هم‌زمان) برای درایورهای پنجره‌ای.

فصل ۷

نتیجه‌گیری و کارهای آینده

۱.۷ نتیجه‌گیری

در این پایان‌نامه یک چارچوب تست سرتاسری متن‌باز برای نرم‌افزارهای مبتنی بر Qt C++ در بستر ویندوز طراحی و پیاده‌سازی شد. چارچوب روی زیرساخت استاندارد Appium/WinAppDriver سوار است و آنچه این زیرساخت نمی‌دهد — پایداری، خوانایی و نگهداشت‌پذیری تست‌ها — را با پنج لایه‌ی معماری می‌سازد: لایه‌ی مکان‌یابی با نهانگاه دسته‌ها و جستجوی محدوده‌ای، لایه‌ی اشیای صفحه، لایه‌ی زبان مخصوص دامنه، کارخانه‌ی درایور با اتصال فازی، و لایه‌ی جداسازی داده و گزارش‌گیری. دو قرارداد پایداری (شناسه پایدار و قرارداد خوانش) رابطه‌ی برنامه‌ی هدف و چارچوب را قواعدمند می‌کند و تستِ نمودارهای گرافیکی را — که در درخت دسترس‌پذیری دیده نمی‌شوند — به تست داده‌ی خوانا تبدیل می‌کند.

ارزیابی روی برنامه‌ی نمونه‌ای پیچیده (جریان ورود چندپنجره‌ای، فیلترهای آبشاری، صفحه‌بندی، مجازی‌سازی جدول، نمودار و گزارش‌گیری) با ۲۵ سناریوی سرتاسری انجام شد؛ اجرای کامل پایدار، بازتولیدپذیر و در حدود شش دقیقه است و گزارش تعاملی Allure با پیوست تصویر خطا تولید می‌شود. جداسازی پایگاه‌داده در سطح تست، استقلال و بازتولیدپذیری سناریوها را تضمین می‌کند و مسیر انتقال به CD/CI به‌صورت طراحی‌شده آماده است. همه‌ی اجزا متن‌باز و بدون هزینه‌ی مجوز هستند.

۲.۷ کارهای آینده

مسیر پیش‌روی چارچوب در چهار جهت است:

- **اجرا در پایپلاین:** برپایی رانر ویندوزی، نهان‌گاه وابستگی‌ها و انتشار خودکار گزارش پس از هر تغییر؛ مسیر اجرای بی‌سر با DRIVER_MODE=direct آماده است.
- **گسترش پلتفرم:** افزودن درایورهای دیگر از الگوی کارخانه (Linux با AT-SPI، مک با Accessibility API) بدون تغییر لایه‌های بالا؛ قراردادهای پایداری مستقل از پلتفرم‌اند.
- **تولید خودکار تست:** استفاده از بازشماری کنترل‌های برنامه (ابزار بازرسی موجود در چارچوب) برای پیشنهاد اسکلت سناریوها و کشف انحراف شناسه‌ها در زمان ساخت برنامه.
- **تحلیل نتایج در گذر زمان:** نگهداری روند زمان اجرا و نرخ شکست اجراها برای آشکارسازی تست‌های شکننده پیش از آنکه تبدیل به مزاحم شوند.

مراجع

[1] OpenJS Foundation, "Appium: Automation for Apps," https://appium.io/docs/en/latest/, 2024. Accessed: 2026-10-20.

[2] Microsoft, "WinAppDriver: Windows Application Driver," https://github.com/microsoft/WinAppDriver, 2023. Accessed: 2026-10-20.

[3] M. Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

[4] G. Meszaros. *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley, 2007.

[5] The Qt Company, "Qt 6.8 Documentation," https://doc.qt.io/qt-6/, 2024. Accessed: 2026-10-20.

[6] SQLite Consortium, "SQLite Documentation," https://www.sqlite.org/docs.html, 2024. Accessed: 2026-10-20.

[7] Qameta Software, "Allure Report Documentation," https://allurereport.org/docs/, 2024. Accessed: 2026-10-20.

[8] L. Crispin and J. Gregory. *Agile Testing: A Practical Guide for Testers and Agile Teams*. Addison-Wesley, 2009.

[9] Microsoft, "UI Automation Overview (Windows Accessibility APIs)," https://learn.microsoft.com/en-us/windows/win32/winauto/uiauto-uiauto-overview, 2023. Accessed: 2026-10-20.

[10] The Qt Company, "Writing Unit Tests with Qt Test," https://doc.qt.io/qt-6/qttest-index.html, 2024. Accessed: 2026-10-20.

[11] pytest-dev, "pytest: helps you write better programs," https://docs.pytest.org/, 2024. Accessed: 2026-10-20.

[12] Selenium Project, "Selenium Browser Automation," https://www.selenium.dev/documentation/, 2024. Accessed: 2026-10-20.

[13] Cypress.io, "Cypress Documentation," https://docs.cypress.io/, 2024. Accessed: 2026-10-20.

[14] Microsoft, "Playwright Documentation," https://playwright.dev/docs/intro, 2024. Accessed: 2026-10-20.

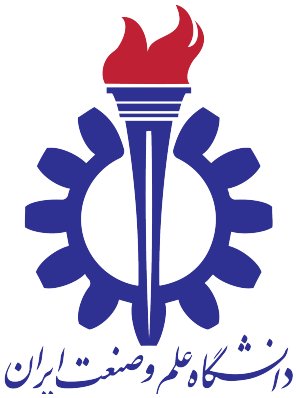
[15] froglogic GmbH, "Squish GUI Test Automation Toolkit," https://www.froglogic.com/squish/, 2021. Accessed: 2026-10-20.

Abstract

Automated UI testing for Qt C++ desktop applications has largely been limited to commercial tools; existing open-source options provide only raw infrastructure (a driver and a protocol) without architectural layers. This thesis presents the design and implementation of an open-source End-to-End (E2E) testing framework for Qt-based Windows desktop applications. The framework builds on WinAppDriver/Appium and adds five architectural layers: element location with a handle cache and scoped finds, page objects, a domain-specific language, a driver factory with phase attachment, and database isolation with reporting. Two stability contracts (stable identifiers and the readout contract) formalize the relationship between the target application and the framework, turning the testing of graphical charts into testing readable data.

For evaluation, a demo application, the Iran Signal Tracker (Qt and SQLite based), was built with deliberately hard-to-test flows: multi-window authentication, cascading filters, pagination, table virtualization, charts, and reporting. Twenty-five E2E scenarios were implemented; the full run is stable, reproducible, and completes in about six minutes, producing an interactive Allure report with failure screenshots. A CI/CD migration path is also designed.

Key Words: End-to-End Testing, UI Automation, Qt, WinAppDriver, Appium, Domain-Specific Language, Page Object



School of Computer Engineering

Design and Implementation of an End-to-End Testing Framework for Qt C++ Applications

Bachelor's Thesis

In the field of Computer Engineering

Alireza BayatKhani

Advisor

Dr. Abolfazl Dianat

Autumn 2026